
Understanding APIs

B.NEEHARIKA

SECTION A · QUESTIONS 1–4

API Fundamentals

Q1. What does API stand for, and what is its primary purpose in software communication?

A: API stands for Application Programming Interface. It is a set of rules and protocols that allows one piece of software to request services or data from another piece of software. Its primary purpose is to let two different applications communicate with each other in a standardized, predictable way — without either side needing to know the internal code of the other.

Q2. Using the restaurant analogy from the session, explain the role of the “waiter” in an API request.

A: In the restaurant analogy, the customer is the client, the kitchen is the server, and the API is the waiter. The customer never talks directly to the kitchen — they tell the waiter what they want (the request), the waiter carries that order to the kitchen, and once the food is ready the waiter brings it back to the customer (the response). The API/waiter acts as the messenger that carries structured requests and responses between two systems that would otherwise have no direct way to interact.

Q3. Give two real-world examples of applications that rely on APIs (other than those covered in the session) and name the type of API each likely uses.

- Uber — uses a Maps/Geolocation API (e.g., Google Maps API, a REST API) to track driver and rider locations and calculate routes.
- Zomato/Swiggy (food delivery apps) — uses a Payment Gateway API (e.g., Razorpay/Stripe, a REST API) to process online payments securely.

Both are examples of third-party REST APIs that let an application plug in a specialized capability instead of building it in-house.

Q4. Why do companies prefer using existing APIs (for maps, payments, login, etc.) instead of building these services from scratch?

A: Building services like maps, payment processing, or authentication from scratch requires enormous engineering effort, ongoing maintenance, security compliance, and domain expertise. Using an existing, proven API saves development time and cost, reduces risk (the provider handles security, uptime, and updates), and lets a company focus its own resources on its core product rather than reinventing infrastructure that specialists already do well.

SECTION B · QUESTIONS 5–7

Client–Server Architecture & Data Travel

Q5. Differentiate between the roles of a “Client” and a “Server” in client-server architecture, with one example of each.

A: A Client is the application or device that initiates a request for data or services — for example, a web browser or mobile app requesting a webpage. A Server is the remote machine that receives that request, processes it,

and sends back a response — for example, the backend system hosting a company's website. The client asks; the server answers and provides the actual resource or data.

Q6. What is the purpose of DNS, and why can't computers communicate using domain names alone?

A: DNS (Domain Name System) translates human-readable domain names (like `www.example.com`) into the numeric IP addresses that computers actually use to locate each other on a network. Computers route traffic using IP addresses, not text names, so a domain name alone cannot be used for communication — DNS acts as a directory/lookup service that converts the name into the address a computer needs before a connection can be made.

Q7. Briefly explain how data physically travels across the internet in packets before reaching its destination.

A: Before being sent, data is broken down into small units called packets. Each packet carries a piece of the data along with header information (such as source and destination IP addresses). These packets travel independently across a series of routers and networks, potentially taking different paths, until they reach the destination device. There, they are reassembled in the correct order to reconstruct the original message — this packet-switching approach makes data transfer efficient and resilient to network congestion or failures.

SECTION C · QUESTIONS 8–11

HTTP & Request–Response Cycle

Q8. Define HTTP and explain why it is described as a “protocol.”

A: HTTP (HyperText Transfer Protocol) is the set of rules that governs how requests and responses are formatted and exchanged between a client and a server over the web. It is called a “protocol” because it defines a common, agreed-upon standard — much like a shared language — that both the client and server must follow so that a request sent by any client can be correctly understood and answered by any server, regardless of what technology each side is built with.

Q9. List the five components that make up an HTTP request.

- URL (the address/endpoint being requested)
- Method (GET, POST, PUT, PATCH, DELETE, etc.)
- Headers (metadata such as content type and authentication)
- Body (the data sent with the request, if any)
- Authentication (credentials/API key proving who is making the request)

Q10. What is the difference between `response.status_code` and `response.json()` when reading an API response?

A: `response.status_code` returns the HTTP status code (e.g., 200, 404, 500) that tells you the outcome of the request — whether it succeeded, failed, or encountered an error. `response.json()` parses the body of the response and converts it from JSON text into a Python dictionary (or list), giving you the actual data returned by the API. In short, `status_code` tells you what happened, while `json()` gives you what was returned.

Q11. Using the pizza-ordering analogy, match each part of an order (URL, Method, Headers, Body, Authentication) to its real-world equivalent.

Order Part	Real-World Equivalent
URL	The pizza shop's address — where the order is being sent
Method	The action being taken — e.g., placing a new order (POST) vs. checking order status (GET)
Headers	Extra instructions on the order slip — e.g., “deliver to the back door,” language/format preferences
Body	The actual order details — pizza size, toppings, quantity
Authentication	Your membership card or phone number that verifies you are a valid, recognized customer

SECTION D · QUESTIONS 12–15

URLs, Endpoints & HTTP Methods

Q12. Break down the URL `https://api.openai.com/v1/chat/completions` into its four components.

Component	Value in this URL
Protocol	<code>https://</code> — specifies secure HTTP is used
Domain/Host	<code>api.openai.com</code> — the server being contacted
Path/Version	<code>/v1</code> — indicates the API version being used
Endpoint	<code>/chat/completions</code> — the specific resource/action being requested

Q13. What is an API endpoint? Give an example of two different endpoints for a shopping application.

A: An API endpoint is a specific URL path within an API that represents a particular resource or function — it's the exact “address” you send a request to in order to perform one specific action. For a shopping application, examples could include `/api/products` (to retrieve or list products) and `/api/orders` (to create or view customer orders).

Q14. Match each HTTP method — GET, POST, PUT, PATCH, DELETE — to its correct purpose.

Method	Purpose
GET	Retrieve/read existing data from the server
POST	Create a new resource on the server
PUT	Replace/update an entire existing resource

Method	Purpose
PATCH	Partially update part of an existing resource
DELETE	Remove an existing resource from the server

Q15. Explain the key difference between PUT and PATCH when updating a resource.

A: PUT replaces the entire resource with the new data provided — any fields not included in the request may be overwritten or removed. PATCH, on the other hand, applies a partial update, modifying only the specific fields included in the request while leaving the rest of the resource unchanged. In short: PUT = full replacement, PATCH = partial modification.

SECTION E · QUESTIONS 16–19

Status Codes, Headers & Authentication

Q16. What do the status codes 401 and 403 indicate, and how are they different from each other?

A: 401 Unauthorized means the request lacks valid authentication credentials — the server doesn't know who you are (e.g., missing or invalid API key). 403 Forbidden means the server does know who you are, but you don't have permission to access the requested resource. In short, 401 is “please authenticate,” while 403 is “you're identified, but you're not allowed here.”

Q17. Why is a 429 status code returned, and what should a well-designed application do when it receives one?

A: A 429 “Too Many Requests” status code is returned when a client has sent more requests than the API's rate limit allows within a given time window. A well-designed application should catch this response and implement a retry strategy with a delay — often called “exponential backoff” — waiting progressively longer between retries, and should also respect any Retry-After header the server provides, rather than immediately re-sending requests.

Q18. What is the purpose of the “Authorization: Bearer <API_KEY>” header?

A: This header is used to authenticate the request by including the client's API key (a secret token) directly in the request headers. The word “Bearer” indicates the authentication scheme being used — it tells the server that whoever holds (bears) this token is authorized to access the API, so the server can verify the key and grant or deny access accordingly.

Q19. List three golden rules for handling API keys securely, as covered in the session.

- Never hard-code API keys directly in source code — store them in environment variables or a secrets manager instead.
- Never commit API keys to version control systems like GitHub, even in private repositories.
- Restrict and rotate keys regularly, giving each key only the minimum permissions it needs (principle of least privilege), and revoke/replace any key that may have been exposed.

SECTION F · QUESTIONS 20–22

JSON — the Language of APIs

Q20. Why is JSON considered the “standard format” for API responses?

A: JSON (JavaScript Object Notation) is lightweight, human-readable, and easy for both people and machines to parse. It is language-independent — nearly every modern programming language can read and generate JSON — which makes it an ideal universal format for exchanging structured data between systems built in different technologies. This wide support and simplicity is why it has become the de facto standard for API request and response bodies.

Q21. Explain one key difference between a Python dictionary and JSON.

A: A Python dictionary is a native in-memory data structure available only within Python code, and it can hold Python-specific values like True/False, None, or even objects and functions. JSON is a text-based, language-independent data-interchange format, and it uses true/false and null (lowercase, no Python types) as its equivalents. In short, a Python dict exists only in Python's runtime, while JSON is a plain-text format that must be “dumped” from or “loaded” into a dictionary to move between systems.

Q22. Convert the following Python dictionary into valid JSON format: {"name": "Employee", "department": "IT", "active": True}

```
{
  "name": "Employee",
  "department": "IT",
  "active": true
}
```

Note: Python's True becomes JSON's lowercase true — this is the key syntax change required for valid JSON.

SECTION G · QUESTIONS 23–26

The Python requests Library**Q23. Which function from the requests library would you use to (a) read data, (b) create new data, and (c) delete data?**

Action	Function
(a) Read data	requests.get()
(b) Create new data	requests.post()
(c) Delete data	requests.delete()

Q24. What is the purpose of the timeout parameter in a requests.get() call, and why is it considered a best practice?

A: The timeout parameter specifies the maximum number of seconds the program should wait for the server to respond before giving up. It is a best practice because, without it, a request could hang indefinitely if the server is slow or unresponsive — freezing the application. Setting a timeout ensures the program fails gracefully and quickly instead of getting stuck waiting forever.

Q25. Write a short Python try/except block that safely calls an API and handles a requests.exceptions.RequestException.

```
import requests
```

```
try:
    response = requests.get(
        "https://api.example.com/data",
        timeout=5
    )
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as e:
    print(f"API request failed: {e}")
```

Q26. What is the advantage of using a `requests.Session()` object over calling `requests.get()` repeatedly?

A: A `requests.Session()` object persists certain parameters — such as headers, cookies, and authentication — across multiple requests, and it reuses the underlying TCP connection instead of opening a new one each time. This makes repeated calls to the same host significantly faster and more efficient, and it reduces the amount of repetitive code needed to set headers or auth on every single call.

SECTION H · QUESTIONS 27–30

Real-World Application & Best Practices

Q27. Describe, step by step, how a prompt typed into an AI chatbot travels to the AI model and back as a response.

- The user types a prompt into the chatbot's client interface (web or mobile app).
- The client packages the prompt into an HTTP POST request, placing the text in the request body as JSON and attaching headers, including the `Authorization: Bearer <API_KEY>` header.
- The request travels across the internet — via DNS resolution and packet routing — to the AI provider's API server (the endpoint, e.g., `/v1/chat/completions`).
- The server authenticates the request, processes the prompt through the AI model, and generates a response.
- The server sends back an HTTP response containing a status code (e.g., 200 OK) and a JSON body with the model's generated text.
- The client parses the JSON response (`response.json()`) and displays the AI's reply to the user.

Q28. List four best practices an employee should follow when working with any company API in a production environment.

- Never hard-code or expose API keys in code, logs, or public repositories — use environment variables or a secrets manager.
- Always set a timeout on requests and wrap calls in try/except blocks to handle failures gracefully.
- Respect rate limits — handle 429 responses with retry/backoff logic instead of hammering the API.
- Validate and log responses (status codes and errors) properly, and use a `Session()` for efficiency when making repeated calls to the same API.

Q29. A colleague hard-codes an API key directly into a Python script and pushes it to GitHub. Identify the risk and explain the correct approach.

A: The risk is that the API key becomes publicly visible in the repository's commit history — even if it's later removed, it often remains accessible in old commits. Anyone who finds it can use the key to make requests on the company's behalf, potentially running up costs, accessing sensitive data, or abusing the service, all billed or attributed to the company. The correct approach is to store the key in an environment variable or a secrets manager, load it into the script at runtime (e.g., `os.environ.get("API_KEY")`), add any credentials file to `.gitignore`, and immediately revoke and regenerate the key if it was ever exposed in a public commit.

Q30. In your own words, summarize the complete journey of an API call — from client request to server response — using the concepts covered in this session.

A: An API call begins when a client application constructs an HTTP request, specifying a URL/endpoint, a method (GET, POST, etc.), headers (including authentication such as an API key), and, if needed, a body containing data in JSON format. DNS resolves the domain name in the URL to an IP address, and the request travels across the internet as packets to reach the destination server. The server receives the request, authenticates it, processes the desired action, and prepares a response — returning a status code (indicating success, error, or another outcome) along with a JSON body containing the requested data. Back on the client side, tools like Python's `requests` library parse this response (`status_code` and `json()`), and the application uses that data to complete its task — whether that's displaying a webpage, showing search results, or returning an AI-generated reply. This request–response cycle, standardized by the HTTP protocol and structured through JSON, is the foundation of how virtually all modern software systems communicate with one another.